Funciones

Las declaraciones de funciones se mueven automáticamente al principio del ámbito
-> pueden usarse antes de la declaración, siempre que estén dentro del ámbito.

```
let name = Ana;
function showName() {
     console.log(name);
}
showName();
```

-> Ana

```
let name = Ana;
showName();
function showName() {
     console.log(name);
}
```

-> Ana

RECURSIVIDAD

Es una solución de programación que debe manejarse con cuidado: No debemos usarla si no podemos estimar el número de llamadas integradas.

Es muy importante no olvidar la condición de parada, que interrumpirá la recursividad de las llamadas

```
function factorial (n) {
     return n > 1 ? n * factorial(n - 1) : 1;
}
console.log(factorial(6));
```

-> 720

Funciones

En JavaScript los parámetros de una función tienen establecido el valor predeterminado ***undefined***

```
function dividir(numerador, denominador = 1) {
  return (numerador/denominador);
}
```

```
console.log(dividir(4));
console.log(dividir(4,2));
console.log(dividir());
```

```
4
2
NaN
```

```
function dividir(numerador =1, denominador) {
  return (numerador/denominador);
}
```

```
console.log(dividir(4));
console.log(dividir(4,2));
console.log(dividir());
```

```
NaN
2
NaN
```

Funciones

En JavaScript, las funciones pueden tratarse como cualquier dato, y almacenarse en variables o pasarse como argumentos a otras funciones.

```
function tarea() {
     return undefined;
}

let a = tarea();
let b = tarea;
console.log(typeof a);
console.log(typeof b);
```

undefined
function

```
function mostrarMsg(mensaje) {
     console.log(`Mensaje: ${mensaje}`);
}
let sm = mostrarMsg;

sm("Hola!");
console.log(typeof sm);
```

Mensaje: Hola!
function

Funciones

```
function suma(a, b) {
     return a + b;
}
function producto(a, b) {
     return a * b;
}
function operacion(func, first, second) {
     return func(first, second);
}
console.log(operacion(suma, 10, 20));
console.log(operacion(producto, 10, 20));
```

-> 30
-> 200

PARÁMETROS VARIABLES

arguments en un objeto *similar* a array disponible en la mayoría de las funciones javascript, que almacena todos los argumentos que se han indicado en la invocación de la función. Permite prescindir en la definición de la función de los argumentos que utiliza.

```
function suma() {

  let sum = 0;
  for (let i=0; i< arguments.length; i++){
    sum += arguments[i];
  return sum;
  }

  x = suma (10,11,12,13,14);
  console.log (x);
}
```

60

```
function func1(a, b, c) {
  console.log(arguments[0]);

  console.log(arguments[1]);

  console.log(arguments[2]);
}

func1(1, 2, 3);
```

1
2
3

PARÁMETROS REST

Otra opción para trabajar con parámetros variables es utilizando el operador de propagación (…)
Este operador convierte en un array la lista de parámetros usados en la invocación.
Solo el último parámetro puede ser un parámetro rest.

```
function myFunction(a, b, ...masArgumentos) {
  console.log("a", a);
  console.log("b", b);
  console.log("otros argumentos:", masArgumentos);
}

myFunction("one", "two", "three", "four", "five", "six");
```

a one
b two
otros argumentos: [three, four, five, six]

FUNCIONES POR EXPRESIÓN (O EXPRESIONES DE FUNCIÓN)

Una función ya declarada se puede almacenar en una variable y utilizarla después.
Pero es posible también juntar en la misma sentencia la declaración de la función y su almacenamiento en una variable.

```
function suma(a, b) {
     return a + b;
}
let miSuma = suma;
console.log(miSuma(10, 20));
```

30

```
let miSuma = function suma(a, b) {
     return a + b;
}
console.log(miSuma(10, 20));
```

30

FUNCIONES ANÓNIMAS – FUNCIONES LAMBDA

Una función por expresión se convierte en una función anónima si se elimina el identificador de función.

```
let miSuma = function suma(a, b) {
     return a + b;
}
console.log(miSuma(10, 20));
```

30

```
let miSuma = function(a, b) {
     return a + b;
}
console.log(miSuma(10, 20));
```

30

Las funciones anónimas no se pueden utilizar hasta que la variable no esté inicializada, como ocurre con en el resto de las variables. Recordemos que las funciones declarativas (funciones comunes) pueden invocarse en el código antes de ser definidas.

CALLBACKS : FUNCIONES DE LLAMADA

Se llaman así a las funciones anónimas que se pasan de argumento a otras funciones.

(1) Se define una función que al ser anónima se vincula a una variable

(2) Se utiliza esa variable como parámetro de otra función que la ejecutará.

```
const welcome = function() {
    return "Bienvenido";
  }

const usuario = function (callback){
  console.log(callback() + "usuario");
}

usuario(welcome);
```

"Bienvenido usuario"

FUNCIONES FLECHA

Una función flecha es una forma simplificada de escribir una función.

- Se elimina la palabra reservada function y se sustituye por =>
- Las llaves se pueden omitir si el cuerpo de la función tiene solo una línea.
- Si se pueden omitir las llaves, el return también se puede obviar.
- En el caso de que la función tuviera un solo parámetro, también se puede prescindir de los paréntesis iniciales.

```
let miSuma = function suma(a, b) {
     return a + b;
}
console.log(miSuma(10, 20));
```

30

```
let miSuma = (a, b) => {
     return a + b;
}
```

```
let miSuma = (a, b) => a + b;
```

EJERCICIO FUNCIONES

Escribe tres funciones con los nombres suma, resta, y multiplica, que aceptarán dos argumentos numéricos. Las funciones comprobarán si los argumentos dados son enteros (usar Number.isInteger()) Si no son enteros devolverán NaN y si lo son devuelven el resultado de la suma, resta o multiplicación, respectivamente.

1. Las funciones deben declararse mediante una instrucción de función
2. Reescribe las funciones una expresión de función de flecha, tratando de escribirlas en la forma más corta posible.

EJERCICIO . CALLBACK

Escribe una función *ejecuta* con tres argumentos: el primero será una función (*callback)* y los otros dos argumentos serán números. Como función de llamada, podrá recibir una de las tres funciones del ejercicio anterior (suma, resta, multiplica). La función ejecuta llamará a la función que se le haya pasado con los argumentos segundo y tercero que ha recibido y devolverá el resultado obtenido.

Se probará así:

```
console.log(ejecuta(suma, 12, 10));          // -> 22
console.log(ejecuta(resta, 12, 10));         // -> 2
console.log(ejecuta(multiplica, 10, 10.1));  // -> NaN
```